Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 7 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Week 7 Overview

The objective of Week 7 was to implement Continuous Integration (CI) for the Automated Grocery Inventory Dashboard using Jenkins.

The main activities completed were:

- Installed and configured Jenkins.
- Connected the GitHub project repository with Jenkins.
- Configured the `develop` branch as the build source.
- Configured Maven-based project compilation and packaging.
- Configured automatic polling using Poll SCM.
- Generated and archived the Spring Boot JAR artifact.
- Verified a successful Jenkins build.

2. CI Pipeline Configuration

The Jenkins Freestyle project was created with the name:

Automated-Grocery-Inventory-CI

The GitHub repository was configured as the source code repository, with the `develop` branch selected for continuous integration.

The build trigger was configured using Poll SCM with the following schedule:

```
H/5 * * * *
```

This allows Jenkins to periodically check the repository for new commits and initiate a build when changes are detected.

The build step uses the Maven Wrapper included in the project:

```
export JAVA_HOME=/opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home
export PATH="$JAVA_HOME/bin:$PATH"

java -version
chmod +x mvnw
./mvnw clean package
```

The generated JAR files are archived using:

```
target/*.jar
```

3. Jenkins Job Configuration

The Jenkins configuration connects the GitHub repository, checks out the `develop` branch, executes the Maven build, and archives the generated artifact.

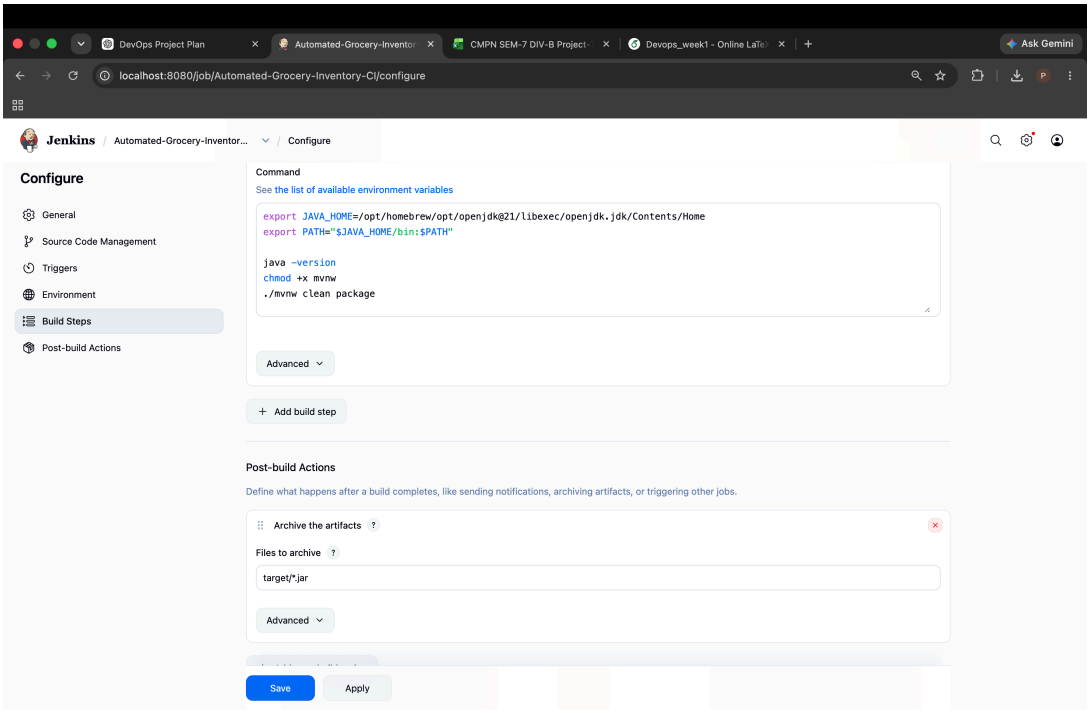


Figure 1: Jenkins CI Job Configuration

4. Successful CI Build

The first Jenkins build initially failed because the Jenkins service could not locate the Java runtime. The Java environment was then explicitly configured using `JAVA_HOME`.

The second build successfully detected Java 21, compiled the project, executed the tests, packaged the application, and completed successfully.

The build result was:

BUILD SUCCESS
Finished: SUCCESS

The project test execution completed with zero failures and zero errors.

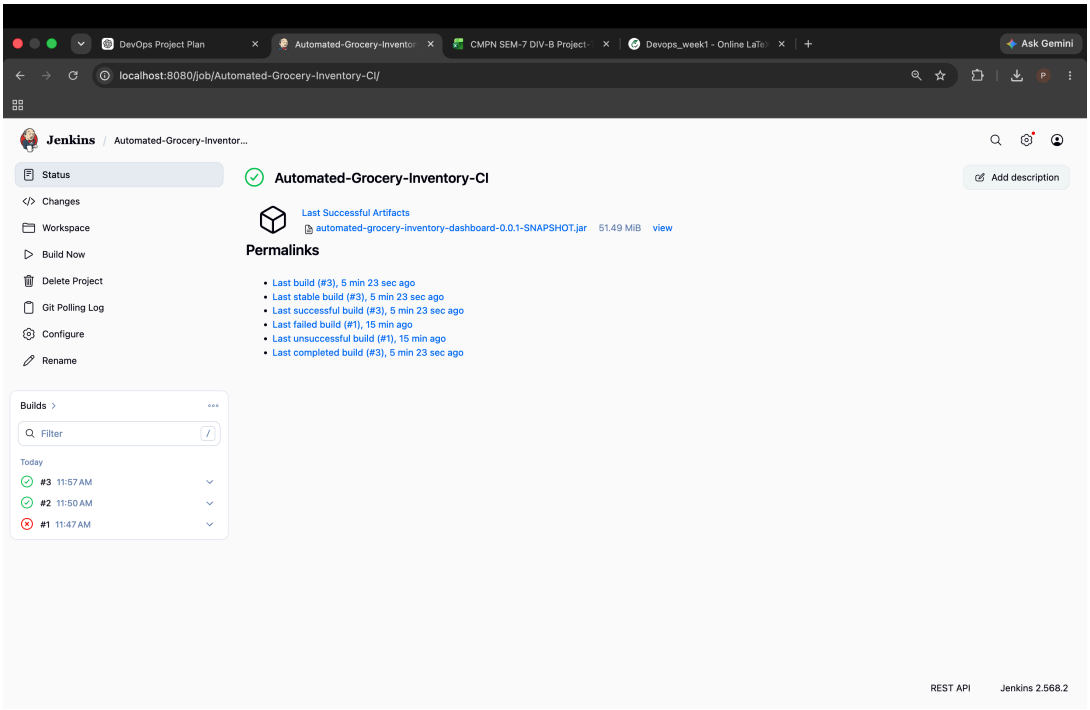


Figure 2: Successful Jenkins Maven Build

5. Artifact Archiving

After the Maven build completed successfully, Jenkins archived the generated Spring Boot application JAR.

The archived artifact was:

`automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.jar`

This provides a ready-to-use build artifact associated with the successful Jenkins build.

6. Automatic Trigger Verification

A test commit was pushed to the `develop` branch to verify the configured Poll SCM trigger.

Jenkins periodically checks the GitHub repository according to the configured polling schedule. After detecting the new commit, Jenkins initiates a new build automatically.

7. Conclusion

Week 7 successfully established Continuous Integration for the Automated Grocery Inventory Dashboard. Jenkins is connected to the GitHub repository, automatically checks for changes, executes the Maven build and tests, and archives the generated JAR artifact.

The Week 7 deliverable was therefore completed with a configured Jenkins CI job, successful build log, automatic trigger verification, and archived build artifact.